Colour Maximite 2

Loadable Programs

Programs in C and C++

For firmware 7.00.00RC5 (services version 18)

Peter Mather

(12 November 2026)

The firmware, this guide and the tools that build a program are at
https://github.com/UKTailwind/CMM2V1.5

About This Guide

This guide is for programmers who want to write programs in C or C++ for the Colour Maximite 2, or to port existing software to it, as loadable programs: files ending .APP that MMBasic runs with RUN "name.app". It covers the tools that build a program, how the firmware loads and runs one, the services that the firmware gives it, and the experience of porting MicroPython, Doom and ScummVM. It assumes a working knowledge of C and of the MMBasic user manual.

Support

Questions should be raised on the Back Shed forum (http://www.thebackshed.com/forum/Microcontrollers), and problems with the tools or the services reported as issues of the repository above.

Copyright

This guide is copyright 2026 Peter Mather and is distributed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Australia license (CC BY-NC-SA 3.0), as the user manual is. The files of the tools (tools/sdk) are under the MIT licence, so a program built with them can have any licence its author chooses (see Appendix C).

Contents

Introduction 5
What Has Been Done This Way 5
What You Need 5
Generation 1 and Generation 2 5
This Guide 6
Getting Started 7
Setting Up the PC 7
A First Program 7
Building It 7
Copying It to the SD Card 7
Running It 7
The Examples 8
How a Program Runs 9
RUN 9
The Memory on a Generation 2 9
The Memory on a Generation 1 9
The Display Memory Lent 10

The Start 10
The Stack 11
The End 11
Ctrl-C 11
The Rules 11
Building Programs 13
build.py 13
How It Compiles 14
The Services' Version 14
C++ 15
Larger Projects 15
Using Another Build System 15
mkimage.py and the Relocations 15
The C Library 17
The Console 17
Files 17
Memory 17
Time 18
Numbers 18
What Is Not There 18
The Services 19
Errors 19
The Console 19
The Clock 19
Files 19
Directories 20
The System 20
The FM Synthesiser 21
MMBasic's Statements and Expressions 21
The Screen 22
Setting a Mode 22
The Palette 23
Waiting for a Frame 23
A Display in the Program's Memory 23
Drawing 24
The Keyboard and Mouse 25
Characters Typed 25
Keys Held Down 25
The Keyboard's Own Codes 26
The Mouse 26
Game Controllers 26
USB Gamepads 26
Sound and Music 27
The Sound Stream 27
Code Called From an Interrupt 27
Music 28

The FM Synthesiser Directly 28
The I/O Pins 29
The Pins 29
Errors 29
Digital and Analog Pins 29
PWM 30
I2C 30
SPI 30
The COM Ports 30
At the End 31
Performance 32
Code and Data in the SDRAM 32
Code in the Fast Memories 32
Floating Point 32
The Interrupts 32
Some Figures 33
Debugging 34
printf 34
When a Program Faults 34
Memory 34
Testing the Moves 34
Running Again Quickly 34
Porting Existing Software 35
The Approach 35
Doom 35
ScummVM 35
MicroPython 36
Things That Catch Ports 36
Appendix A – The Services 37
Appendix B – The Program File 40
The Header 40
The Image 40
The Relocations 40
Appendix C – Licences 41
The Tools 41
Code Beside the Examples 41
The Programs Included with the Firmware 41
In the Firmware 41

Introduction

A loadable program is a program written in C or C++, compiled on a PC for the Colour Maximite 2's processor and run from the SD card or a USB flash drive with MMBasic's RUN command:

```
RUN "A:/apps/hello.app", any words
```

It runs at the full speed of the processor, an STM32H743 (an Arm Cortex-M7 at 480MHz, or 400MHz with the earlier revision of the chip that many Generation 1 boards have, with a double precision floating point unit). MMBasic stays where it is in flash and the program reaches the computer's hardware through a table of services that the firmware gives it: the console, the clock, the files on the SD card and USB flash drive, the screen, the keyboard and mouse, the sound output, an FM synthesiser and the I/O pins. When the program ends MMBasic tidies up after it and carries on: at the command prompt it returns to the prompt, and in a BASIC program it goes on with the next statement, with the BASIC program and its variables as they were.

This makes the Colour Maximite 2 a platform for software that would be too slow or too large in BASIC, and for software that already exists in C: games, emulators and other programming languages.

What Has Been Done This Way

Three large programs are included with the firmware, and they show what is possible:

- **MicroPython**, the Python 3 of microcontrollers, as on the Pico Computer 3: its display, keyboard, mouse, sound and image modules, SQLite and ulab (a NumPy for MicroPython), and the I/O pins through the machine module. It runs Python about twice as fast as the RP2350 does.
- **Doom**, through doomgeneric, with the Freedoom game data: the original 320 x 200 game with its sound effects and its music on the FM synthesiser, at about 50 frames a second on a Generation 2 and 35 on a Generation 1.
- **ScummVM**, the interpreter for the classic adventure games of LucasArts, Sierra and others (from frank-quest, its port to the RP2350): the AGI, SCI, SCUMM, Gobliiins and Kyrandia engines, with music and speech.

Each was ported with the tools and services described in this guide, and the chapter Porting Existing Software describes how.

What You Need

- A Colour Maximite 2, Generation 1 or 2, with firmware 7.00.00RC0 or later.
- A PC running Windows, Linux or macOS with Python 3 and the Arm GNU toolchain for bare metal targets (arm-none-eabi-gcc). The firmware is built with GCC 13.3 (ST's or Arm's own) and that is the version to use.
- The firmware's repository, which holds the tools (tools/sdk), the header that describes the services (Inc/cmm2img.h) and the example programs: `git clone https://github.com/UKTailwind/CMM2V1.5`
- A way to put files on the SD card: a card reader, or XMODEM or ZMODEM over the serial console.

Generation 1 and Generation 2

The same program runs on both generations. The differences that matter to a program are its speed and its memory:

	Generation 1	Generation 2
Processor clock	400MHz or 480MHz, by the chip's revision	480MHz
Memory for the program	about 4MB, borrowed from MMBasic's heap while the program runs; about 7MB with the display memory too, for a program built with --vram	8MB set aside for it
Where the program is loaded	where the free memory is: it is moved there	0xD1800000, where it was linked

Video modes	as for BASIC: no MODE 16 and no 32-bit colour; with --vram, those whose page fits in the internal RAM	all

A program that needs more memory than a Generation 1's 4MB is built with the --vram option: a Generation 1 then lends it the display memory as well while it runs, and limits it to the video modes that do not need it (see The Display Memory Lent in the next chapter). Doom and ScummVM are built this way, and the same file runs on both generations. A program that cannot run on a Generation 1 at all is built with --g2, and a Generation 1 then refuses it with the error "Needs a G2".

This Guide

The chapter Getting Started builds and runs a first program. How a Program Runs describes what the firmware does with a program from RUN to its end, and Building Programs the tools that make one. The C Library covers the standard C functions a program can use. The Services and the four chapters after it describe the services, with examples: the screen, the keyboard and mouse, sound and music, and the I/O pins. The last chapters cover performance, debugging and porting existing software. The appendices list the services, describe the program file's format and give the licences.

Getting Started

This chapter builds a first program, puts it on the SD card and runs it.

Setting Up the PC

Install Python 3 (3.8 or later) and the Arm GNU toolchain for arm-none-eabi, version 13.3: Arm's own (Arm GNU Toolchain 13.3.Rel1, from developer.arm.com) or the one that comes with ST's STM32CubeIDE. Then get the firmware's repository:

```
git clone https://github.com/UKTailwind/CMM2V1.5
```

The tools are in its tools/sdk directory. They find the compiler in the folder named by the environment variable ARM_TOOLCHAIN_DIR, if it is set, or else through the PATH. On Windows, for example (with the folder that holds arm-none-eabi-gcc.exe):

```
setx ARM_TOOLCHAIN_DIR "C:\Arm GNU Toolchain\13.3 rel1\bin"
```

A First Program

Make a file called first.c in tools/sdk:

```
#include <stdio.h>
#include "cmm2.h"

int main(int argc, char **argv) {
    int i;
    printf("Hello from a loadable program at %lu MHz\n",
           (unsigned long)(cmm2->cpu_hz() / 1000000));
    for(i = 0; i < argc; i++)
        printf("argv[%d] = %s\n", i, argv[i]);
    printf("My heap: %lu KB\n", (unsigned long)(cmm2_start->heap_size / 1024));
    return 0;
}
```

It is ordinary C. printf() goes to the console: the serial port and the screen, as MMBasic's PRINT does. cmm2.h gives the program two variables: cmm2, the table of the firmware's services (here its cpu_hz service), and cmm2_start, what the firmware passed to the program as it started it (here the size of its heap).

Building It

In tools/sdk:

```
python build.py out/first.app first.c
```

This prints:

```
compiling 3 of 3 sources, 24 at a time
out/first.app: 12540 bytes of image at 0xD1800000, 348 of .bss, entry +0x1, 152
relocations (1 absolute left alone), CRC DB38C447
```

build.py compiled first.c with two files of the tools, crt0.c (the program's start) and syscalls.c (what joins the C library to the services), linked them for the address where a Generation 2 loads a program, and wrote the program file out/first.app (the sources are compiled in parallel, as many at a time as the PC has processor threads). The ELF file and a map of the program are beside it (out/first.elf and out/first.map), for debugging.

Copying It to the SD Card

Copy out/first.app to the SD card with a card reader, or send it over the serial console. To send it with ZMODEM, type this at the Colour Maximite 2's command prompt:

```
ZMODEM RECEIVE
```

and then send the file from your terminal emulator (in Tera Term: File, Transfer, ZMODEM, Send). The repository also has a small Python sender: `python tools/zmodem_pc.py send COM4 115200 tools/sdk/out/first.app` (with your serial port and baud rate). XMODEM RECEIVE "first.app" works too.

Running It

At the command prompt:

```
RUN "first.app", hello world
```

The program prints:

```
Hello from a loadable program at 480 MHz
argv[0] = first.app
argv[1] = hello
argv[2] = world
My heap: 8115 KB
```

and MMBasic's prompt returns. On a Generation 1 the heap is about 4MB, at 400MHz. Change the return statement to return 3, build and run it again, and MMBasic prints "first.app ended with code 3": a program's exit code is shown when it is not zero.

The Examples

tools/sdk/examples has programs to read and build on:

Example	What it shows
hello.c	The console, the arguments, the heap, the clock, files (written, read back and removed), a directory listing, timing, and leaving from inside a function with leave().
vidtest.c	The screen and the keyboard: a video mode, 256 colours from the palette, timing frames, turning the palette and reading the keys held. Built with --api 2.
nopoll.c	What happens to the USB devices when a program does not call poll().
oplbench.c	The firmware's FM synthesiser against Nuked-OPL3 compiled into the program (LGPL: see Appendix C).
oplplay	Renders MIDI, IMF, DRO and VGM music to a WAV file through three different OPL emulators, timing each.

They are built in the same way, for example:

```
python build.py out/hello.app examples/hello.c
python build.py out/vidtest.app examples/vidtest.c --api 2
```

The repository's apps directory holds the build scripts of the larger programs: apps/doom/make.py and apps/scummvm/make.py fetch the original source code and build doom.app and scummvm.app from it with the files in their directories.

How a Program Runs

This chapter follows a program from RUN to its end: how the firmware loads it, the memory it has, how it starts and stops, and what MMBasic puts back afterwards.

RUN

`RUN "name.app" [, words]` runs a loadable program. MMBasic looks at the name: only a name that ends in .APP (in any case) is taken as a loadable program, so RUN of a BASIC program is unchanged. The name can be a string expression (`RUN f$`), and it is found as any file is: on the current drive and in the current directory unless it says otherwise. The BASIC program in program memory is not touched.

Before loading, MMBasic stops any sound that is playing. It then reads the file's header and checks it:

Error	Why
Not a loadable program	The file does not start as a program file does.
Not a program this firmware can load	The header is of another version, or the file's size does not match it.
Needs a newer firmware	The program uses a later version of the services than this firmware has (its --api option).
Needs a G2	A program built with --g2, on a Generation 1.
Needs n KB, n KB free in one block	Generation 1: its code, data, stack and least heap do not fit in the memory that is free. The figure is the largest block.
Needs n KB, more than the 8 MB	Generation 2: they do not fit in the 8MB.
Cannot lend the display memory	Generation 1, a program built with --vram: the screen could not be changed to free the display memory (MMBasic has a frame buffer of another colour depth).
The program file is damaged	The CRC of the code and the relocations does not match the header's.

These are ordinary MMBasic errors, so a BASIC program can catch them with ON ERROR SKIP and MM.ERRNO.

The program's code and data are read straight into their place. If that is not the address the program was linked for, the program is moved: each address in it is adjusted by the difference, using the list of them in the file (the relocations: see Appendix B). Its .bss (its variables without initial values) is cleared to zero, the caches are made to agree with the memory, and the program is called.

The Memory on a Generation 2

A Generation 2 sets aside the top 8MB of its SDRAM, 0xD1800000 to 0xD1FFFFFF, for loadable programs. A program is linked for 0xD1800000, so it is loaded there and nothing in it is moved:

Address	What is there
0xD1800000	The program: its code, constants and initialised data (the image).
after the image	Its .bss, cleared to zero.
after the .bss	Its heap, for malloc() and new: everything up to the stack.
the top 64KB	Its stack (the default: --stack sets another size).

The heap's address and size are in cmm2_start->heap and cmm2_start->heap_size. For a small program the heap is a little under 8MB.

The memory is cached and executable, and runs at the SDRAM's speed through the processor's 16KB instruction cache and 16KB data cache (see the chapter Performance).

The Memory on a Generation 1

A Generation 1 has 8MB of SDRAM in all, most of it used by the display, so it cannot set any aside. Instead a program borrows the largest free part of MMBasic's heap while it runs: all of it but 256KB, which is left for the services and MMBasic. The program is put at the first 64KB boundary in it and moved there by its relocations,

and that part of the heap is made executable while the program runs. When the program ends it is given back and the heap is as it was.

At the command prompt this gives a program about 4.2MB (MicroPython has a 2.3MB heap after its own code and buffers); run from a BASIC program it is what the BASIC program's variables leave free. So a program should never assume the size of its memory: it should size its buffers from cmm2_start->heap_size, as Doom sizes its zone and MicroPython its heap and its SQLite pool.

With OPTION COMPILE ON or SHADOW, the 1MB kept for the compiled BASIC program, just below the heap, is lent as well when no compiled program is running. The next RUN of a BASIC program compiles it again. A program run from a compiled BASIC program leaves that memory alone, and has 1MB less.

The program file says the least heap the program can run with (build.py's --heap option). If the memory there is cannot give it, RUN refuses the program before loading it ("Needs n KB, n KB free in one block"), rather than the program failing later.

The Display Memory Lent

A program built with --vram (version 8) is lent the display memory as well: the 3MB at 0xD0000000 to 0xD02FFFFF that holds MMBasic's video pages. As the program starts, the screen is changed to MODE 7 (320 x 240), whose page is in the processor's internal RAM, so nothing is drawn in the display memory while the program has it.

The display memory and the bottom of MMBasic's heap are normally next to each other, and the program then has the two as one area, laid out as on a Generation 2: its image at 0xD0000000, then its .bss, its heap and its stack. At the command prompt that gives a small program a heap of about 7.2MB. ScummVM, whose image is 3MB, has about 4.2MB of heap, and Doom a 6MB zone.

With OPTION RAM or OPTION COMPILE ON, MMBasic keeps memory between the two, and the program has two blocks instead. It is loaded into the display memory if its image, .bss and stack fit there, and into the borrowed part of MMBasic's heap if not. Each block's free memory is a heap: the larger is cmm2_start->heap, which malloc() uses, and the other is cmm2_start->heap2 (with its size in heap2_size, or NULL if there is none). A program that needs one large buffer can take it from heap2; one that knows nothing of heap2 just has less memory.

While the program runs, the screen can be set only to a mode whose page fits in the internal RAM's 512KB, and video_mode and video_scaled return -1 for the others. In 8-bit colour that is every mode but 9, 11, 12, 15, 16, 18 and 19; in 16-bit colour, modes 2 to 7, 13 and 17. video_scaled at 2 halves a page, so in 8-bit colour MODE 9, 11 and 18 can be used at half their width and height. Doom (MODE 3) and ScummVM (MODE 7) are not limited by this.

The display memory is executable while the program runs. When the program ends it is cleared and made non-executable again (MMBasic keeps it that way, for the fastest drawing), and MMBasic's video mode is set again. On a Generation 2 --vram changes nothing: the program has its 8MB.

The Start

The firmware calls the program's entry point, _start in crt0.c, on the program's own stack, with the start block:

```
typedef struct {
    const struct cmm2_services *svc;    /* the services */
    int      argc;
    char   **argv;                      /* the file's name, then RUN's words */
    void    *heap;                      /* .bss to stack, 8-byte aligned */
    uint32_t heap_size;
    uint32_t base;                      /* where the program was loaded */
    void    *heap2;                     /* a second heap, or NULL (version 8) */
    uint32_t heap2_size;
} cmm2img_start;
```

heap2 is only ever set for a program built with --vram, on a Generation 1 (see The Display Memory Lent above). Firmware before version 8 has a shorter start block, so a program reads heap2 only when it was built with --api 8 or later.

_start keeps the services in the variable cmm2 and the start block in cmm2_start, runs the C++ constructors (and C functions marked constructor), and calls main(argc, argv).

argv[0] is the program's name as RUN was given it ("first.app" or "A:/apps/first.app"). The words after the comma follow, split at spaces; a word in double quotes can hold spaces. The program's current directory is MMBasic's, so a program run with `CHDIR "A:/Doom" : RUN "doom.app"` finds its files in A:/Doom without a path. Doom, for one, looks for its game data beside its own file, using argv[0].

The Stack

A program runs on a stack of its own at the top of its memory (the processor's process stack), 64KB unless build.py's --stack option says otherwise. The firmware's interrupts run on MMBasic's stack, in the processor's fast internal memory, so they do not use the program's. Nothing checks for the stack running over: a stack that grows past its size overwrites the top of the heap without any error. A program with deep recursion or large local arrays should ask for a larger stack.

The End

A program ends by returning from main(), by calling exit(), or by calling the leave service from any depth:

```
cmm2->leave(code);
```

Returning from main() and exit() run the C++ destructors and the functions registered with atexit() and flush the C library's files. leave() goes straight back to MMBasic from wherever it is called (it is the program's longjmp to the firmware), without them. In each case the value is the program's exit code, and MMBasic prints "name.app ended with code n" if it is not zero.

MMBasic then tidies up after the program:

- a frame hook is removed and the DMA2D's copies stopped;
- the I/O pins the program set up are returned to unconfigured, its PWM outputs stopped and its I2C and SPI buses and COM ports closed;
- its sound and music are stopped;
- its files and directories are closed, and the blocks it took from MMBasic's heap with the alloc service are freed;
- on a Generation 1 the memory it borrowed is given back, and the display memory, if it was lent, is cleared and made non-executable again;
- the keys typed while it ran are thrown away, so they are not taken as commands;
- if it changed the video mode, MMBasic's mode, font and colours are set again, and the palette (the 256 colours of an 8-bit mode) is put back as MMBasic had it.

The program's memory is not cleared: a program run next starts with its own image and a cleared .bss.

Ctrl-C

Ctrl-C, from the serial console or a USB keyboard, sets MMBasic's break flag. From then on the poll service returns 1, and the program should end. When it has ended, MMBasic stops the BASIC program that ran it too (as Ctrl-C stops a BASIC program), so a BASIC loop that runs programs one after another is stopped as well.

A program that uses Ctrl-C itself (MicroPython turns it into a KeyboardInterrupt) calls break_take instead, which returns 1 once for each Ctrl-C and clears the flag, so that the BASIC program carries on afterwards.

The serial console's Ctrl-C is seen at once. A USB keyboard's is seen only when the program calls poll, as are its other keys.

The Rules

A loadable program runs with nothing between it and the hardware: it can read and write any memory and any peripheral. That is what makes it fast, and it means that a mistake is not caught as an error. A fault (a bad pointer, a jump to nowhere) restarts the Colour Maximite 2, as a fault in the firmware does. The BASIC program in program memory, the options and the files on the card are not affected. So that the program and MMBasic work together:

- **Use the services** for the hardware MMBasic uses: the console, the files, the screen, the keyboard and mouse, the sound and the I/O pins. They keep MMBasic's view of the hardware right, and MMBasic undoes what they did when the program ends.
- **Call poll() often**, at least every 10 to 20ms (each frame of a game is right). The USB devices, the keyboard and the mouse included, are looked after only when poll() is called, and a USB keyboard's Ctrl-C is seen then.
- **Leave MMBasic's memory alone.** The program's own memory is its image, its heap and its stack, and the blocks it takes with the alloc service. MMBasic's variables, its heap and the internal RAM belong to MMBasic.

- **Leave the processor's set-up alone**: the interrupt vectors, the interrupt priorities, the clocks, the MPU and the caches. The DMA2D is the exception: the firmware does not use it while a program runs, and the dma2d_copy service drives it. On a Generation 2 the AXI RAM and the ITCM above 0xA000 are too, through cmm2_axi_load() (build.py's --axi and --itcm), which makes the AXI RAM cached and executable for the run, and cmm2_axi_unload(), which puts it back.
- **Keep code that runs in an interrupt short**, and call no services from it but the DMA2D's (see Code Called From an Interrupt in the chapter Sound and Music).

Building Programs

A program file is made in three steps: the sources are compiled for the Cortex-M7, linked with the tools' start-up code and library glue at the program's address, and the linked ELF file is turned into a program file. build.py does all three; this chapter describes it, its options, and how to do the same from another build system.

build.py

```
python build.py OUT.app SOURCE.c|.cpp [...] [options]
```

From tools/sdk, or with a path to it. Each source is compiled to an object file in a directory named after OUT (out/first.obj for out/first.app), and the objects are linked together with crt0.c and syscalls.c. The files it leaves are:

File	What it is
OUT.app	The program file, for the SD card.
OUT.elf	The linked program, with its symbols, for debugging (arm-none-eabi-addr2line, objdump, gdb).
OUT.map	The linker's map: where each function and variable is, and what took the space.
OUT.obj/	The object files.

A source is compiled again only when it, or a header it includes, has changed since its object was made (or the options have changed), and the sources are compiled in parallel, so building again after a small change is quick. ScummVM's 531 source files compile in about 23 seconds on a fast PC. An object file (.o) among the sources is linked as it is: a part compiled and linked on its own first, as each of the C compiler's passes is (apps/cc/make.py).

The options:

Option	Meaning
--api N	The version of the services the program needs (default 1). Firmware with an older version refuses the program with "Needs a newer firmware". See below.
--heap BYTES	The least heap the program can run with (default 0). RUN refuses the program at once if it cannot have it.
--stack BYTES	The size of the program's stack (default 64KB).
--vram	A Generation 1 lends the program its display memory too, about 7MB in all, and limits its video modes while it runs (see The Display Memory Lent in the chapter How a Program Runs). Use it with --api 8: firmware before version 8 ignores it. A Generation 2 ignores it too.
--g2	The program runs on a Generation 2 only: a Generation 1 refuses it with "Needs a G2".
--axi NAME	The code, constants and data of the source NAME (its file name without folder or extension) linked to run in the processor's 512KB of AXI RAM, which a Generation 2's display leaves free when its page is in the SDRAM. cmm2_axi_load() copies them there. Repeat it for more sources. Needs --g2. See Code in the Fast Memories in the chapter Performance.
--itcm NAME[:F1,F2,...]	The functions named of the source NAME, or all its code if none are named, linked to run in the ITCM from 0xA000 to 0xFFFF, the 24KB above the firmware's own code, also copied there by cmm2_axi_load(). Needs --g2, and --api 18: older firmware does not keep that memory free.
--libc full	The whole C library, newlib, in place of newlib-nano: its wide-character printf and file functions (swprintf, fwprintf, fgetws), which newlib-nano leaves out. The program is larger: hello.c is 27KB in place of 16KB. Neither library converts UTF-8 as STM32CubeIDE's tools build them: setlocale() takes only "C", and a byte is a character, as in Latin-1.
--cflag FLAG	A flag for the compiler, for C and C++ (eg, --cflag=-DDEBUG, --cflag=-O3). Repeat it for more.

--cxxflag FLAG	A flag for C++ sources only.
--ldflag FLAG	A flag for the link.
-I DIR	A directory of header files.
--sources FILE	A file listing sources, one a line (lines starting with # are left out), for projects too large for a command line.
-j JOBS	How many sources to compile at a time (default: the PC's processor threads).
--base ADDR	Link at another address (default 0xD1800000). Used to test the relocations: see below.

How It Compiles

The flags are those of the firmware, for the STM32H743:

```
-mcpu=cortex-m7 -mfpu=fpv5-d16 -mfloat-abi=hard -mthumb --specs=nano.specs
-O2 -Wall -ffunction-sections -fdata-sections
-fno-exceptions -fno-unwind-tables -fno-asynchronous-unwind-tables
```

C is compiled as gnu11 and C++ as gnu++14, without exceptions or run-time type information (-fno-rtti), and without the locks for local static variables (-fno-threadsafe-statics). The floating point unit does single and double precision in hardware, and float and double are both fast; long double is double.

--specs=nano.specs, which selects newlib-nano's headers and library, is left out with --libc full. The link uses the C library, with printf's floating point included, the maths library and libgcc:

```
-nostartfiles -T app.ld -Wl,--defsym=__app_base=0xD1800000
-Wl,--gc-sections -Wl,--emit-relocs -u _printf_float -lc -lm -lgcc
```

--gc-sections leaves out every function and variable that nothing uses, and --emit-relocs keeps the information that mkimage.py needs to make the relocations. A program with a C++ source is linked by g++, so that the C++ library is there.

The Services' Version

The table of services only grows: each version of the firmware keeps the entries of the one before and may add more at the end. Its version is in cmm2->version:

Version	Added
1	The console, the clock, files and directories, poll, leave, heap blocks, the FM synthesiser.
2	The screen (video_mode, video_palette, video_wait) and the keys held.
3	The sound stream and the music.
4	The mouse, and the console kept off the screen (con_screen).
5	Ctrl-C taken by the program (break_take), and a file's details (file_stat).
6	A display in the program's own memory (video_scaled, frame_hook), the DMA2D and 16-bit colour.
7	The I/O pins, the ADC, PWM, I2C, SPI and the COM ports.
8	No new services: the start block's second heap (heap2), and a Generation 1's display memory lent to a program built with --vram.
9	The USB keyboard's own codes (keys_raw), and fast seeking in a file that is read and written in place.
10	Drawing by MMBasic's routines (gfx_cls to gfx_save), the screen saved as a PNG, and the current directory (dir_change, dir_current).
11	The firmware's own memory routines and maths library (mem_copy, mem_fill, mem_move, libm), and MMBasic's statements and fonts (basic_cmd, gfx_target, font_size, font_define).
12	MMBasic's expressions (basic_eval), what is playing (sound_state), a statement tokenised once (basic_prep, basic_exec), and IMAGE RESIZE_FAST (image_resize).
13	USB gamepads (gamepad).
14	The com_ services reach COM4 to COM7, the USB serial devices.
15	Whether a port's device is there (com_connected).

16	All of a gamepad's axes: both sticks and the analogue triggers (gamepad_axes).
17	video_mode and video_scaled take depth 12: ARGB4444 on one layer.
18	A DMA2D copy without writing the source back from the cache first (dma2d_copy_nc). The ITCM from 0xA000 is kept for programs (--itcm), and the sound stream goes to the DACs by DMA.

Firmware 7.00.00RC3 has version 15, 7.00.00RC4 version 16, and 7.00.00RC5 version 18. A program that uses a service added after version 1 should be built with --api set to the highest version it uses, so that older firmware refuses it with a clear error rather than calling an entry it does not have. A program that can do without a service can instead check the version itself and work with less:

```
if(cmm2->version >= 4) cmm2->con_screen(0);
```

C++

C++ works as it does on any small system without exceptions: classes, templates, virtual functions, new and delete (through malloc), global objects with constructors and destructors, and the parts of the C++ library that do not throw. As nothing can catch an exception, a new that fails ends the program: a program that must survive running out of memory uses new (std::nothrow), malloc(), or an operator new of its own, as ScummVM's port does. ScummVM, a large C++ program, is built this way.

Larger Projects

Point build.py at the sources, the include directories and the flags the project needs, with a --sources file if the list is long. Two things are worth knowing:

- **File names.** On Windows file names are not case sensitive, so a project's header can be found in place of a system or SDK header of the same name (ScummVM's audio.h and the firmware's Audio.h met this way). The tools put only tools/sdk on the include path, and cmm2.h includes the firmware's header by its path.
- **Replacing files.** A port usually replaces a project's platform layer with its own. ScummVM's make.py keeps its replacements in an overlay directory, at the same paths as the files they replace, and builds the project's own source lists with those files swapped in, so the original source stays untouched (see the chapter Porting Existing Software).

Using Another Build System

A project with its own make files can be built without build.py. It needs:

- the compiler flags above, with -I for tools/sdk;
- crt0.c and syscalls.c from tools/sdk, compiled with the same flags;
- the link flags above, with the linker script tools/sdk/app.ld;
- then mkimage.py to make the program file from the ELF.

MicroPython is built this way, by its own Makefile. In outline, its link and program file steps are:

```
LDFLAGS = $(CFLAGS_MCU) --specs=nano.specs -nostartfiles -T $(CMM2_SDK)/app.ld \
    -Wl,--defsym=__app_base=0xD1800000 -Wl,--gc-sections -Wl,--emit-relocs
$(BUILD)/micropython.elf: $(OBJ) $(SDK_O)
    $(CC) $(LDFLAGS) -o $@ $^ $(LIBS)
$(BUILD)/micropython.app: $(BUILD)/micropython.elf
    $(PYTHON) $(CMM2_SDK)/mkimage.py $< $@ --name micropython --api 7 \
        --stack 262144 --heap 1572864
```

mkimage.py takes the options --name (up to 15 characters, the file's name by default), --api, --stack, --heap, --vram and --g2, with the same meanings as build.py's.

mkimage.py and the Relocations

mkimage.py writes the program file: a 64-byte header, the image (every section with contents, from the link address, gaps filled with zeros), and the relocations, a list of the places in the image that hold an absolute address of the image (Appendix B gives the format).

A program linked at 0xD1800000 runs on a Generation 2 without them. A Generation 1 loads the program wherever its free memory is and adds the difference to each of those places. It always moves a program by a whole number of 64KB, so that of the two halves of an address loaded by a MOVW and MOVT instruction

pair only the upper one (the MOVT) changes: mkimage.py keeps the 32-bit addresses (R_ARM_ABS32 and R_ARM_TARGET1) and the MOVTs (R_ARM_THM_MOVT_ABS), and leaves out the relative relocations, which a move does not change. Any other kind of relocation stops the build with an error, so a program that builds will load.

The compiler normally makes only these kinds. Hand-written assembly that loads an absolute address in another way (an LDR of a literal that is not marked as an address) would not be adjusted. To test that a program moves correctly, build it with --base 0x60000000: every address in it must then be relocated, and one that was missed points at memory that does not exist on the Colour Maximite 2 and faults at once.

The C Library

A loadable program has the standard C library (newlib-nano, or the whole newlib with build.py's --libc full) and the maths library. syscalls.c joins them to the firmware's services, so the usual functions work: printf and the rest of stdio, files through fopen, malloc and free, time(), and the functions of string.h, stdlib.h, ctype.h and math.h. This chapter says how each part behaves on the Colour Maximite 2.

The Console

stdout and stderr go to the console: the serial port and the screen, as MMBasic's PRINT does, with each newline sent as a carriage return and line feed. stdout is buffered by the C library until a newline (or fflush(stdout)). A program with a display of its own calls `cmm2->con_screen(0)`, after which the console's output goes to the serial port only and is not drawn over the program's picture.

stdin reads the keyboard: the serial console and a USB keyboard (whose keys come with MMBasic's codes: see Appendix I of the user manual). A read waits for a key, looking after the USB devices while it waits, shows the key, and gives the Enter key as a newline. There is no line editing: a Backspace is read as the character 8. So scanf() and fgets() on stdin work for simple input, and a program that wants keys as they are pressed, without waiting, uses the services: con_getc for typed characters and keys_held for the keys held down (see the chapter The Keyboard and Mouse).

Files

fopen(), fread(), fwrite(), fprintf(), fgets(), fseek(), ftell(), fclose(), remove(), rename() and mkdir() work on the SD card and the USB flash drive. File names are MMBasic's:

Name	Is
A:/dir/name.txt	On the SD card.
C:/name.txt	On the USB flash drive.
/dir/name.txt	From the root of the current drive.
name.txt or dir/name.txt	In or below the current directory (MMBasic's, as CHDIR set it).

The file system is FAT (FAT16, FAT32 or exFAT) with long names, and names are not case sensitive. Up to eight files can be open at once. rename() replaces a file that is there already, as POSIX's does. A file opened for reading, or for reading and writing in place (without creating or appending), is given a map of where its parts are on the card, so that seeking in it is fast: Doom reads its 28MB game data file by seeking to each part, and the 386 PC emulator seeks in its disk images. A write or seek past the end of the file drops the map, as the file then grows.

The POSIX calls under stdio work too: open(), creat(), read(), write(), lseek() and close(), with descriptors 0, 1 and 2 the console and 3 upwards the files. dup2(fd, 0), dup2(fd, 1) or dup2(fd, 2) puts a file in the console's place as standard input, output or error, as a shell's < and > do, so that a program written as a Unix filter reads and writes files; closing that descriptor gives the console back. The C compiler's passes run that way.

opendir(), readdir() and closedir() list a directory, with ".." first below a drive's root, as Linux gives it. stat() gives a file's size, kind and date, rmdir() removes an empty directory, and chdir() and getcwd() are MMBasic's CHDIR and CWD$. The services' file functions (see the chapter The Services) can be used directly too, without stdio's buffering.

A call that fails sets errno: ENOENT for a name or path that is not there, EEXIST for one that is (mkdir() of a drive's root too, as POSIX has it), EACCES for a file that is read-only or a directory that is full or not empty, EROFS for a write-protected card, EMFILE when eight files are open, ENODEV for a drive with no card or no file system, and EIO for the card itself.

Memory

malloc(), calloc(), realloc(), free(), and C++'s new and delete use the program's heap: all the memory between its .bss and its stack (cmm2_start->heap and cmm2_start->heap_size). malloc() returns NULL when the heap is used up.

A program that has its own memory manager, or needs one large block, takes it from the top of the heap with:

```
void *cmm2_heap_take(uint32_t size);
```

which returns the block (8-byte aligned) or NULL, and leaves malloc() the rest. Call it before malloc() has used much of the heap. MicroPython takes its garbage-collected heap this way, and ScummVM all of the heap but

512KB for its allocator to manage. A second heap that a Generation 1 may give a program built with --vram (cmm2_start->heap2: see The Display Memory Lent in the chapter How a Program Runs) is not used by malloc(): it is the program's to use as it likes.

The alloc and release services give blocks of MMBasic's heap instead, outside the program's memory, up to 64 of them. They are freed when the program ends. A program rarely needs them; the FM synthesiser's state (opl_new) is kept in one.

Time

time() and gettimeofday() read MMBasic's clock (DATE$ and TIME$) the first time, and add the microseconds since, so that they never go back. localtime() and strftime() work, with no time zone: the clock's time is taken as local. clock_gettime(CLOCK_MONOTONIC), clock() and times() count the microseconds since the computer started, and nanosleep() waits as usleep() does. For timing, the micros service is simplest: a 64-bit count of microseconds since the computer started.

```
uint64_t t0 = cmm2->micros();
work();
printf("%lu us\n", (unsigned long)(cmm2->micros() - t0));
```

delay_us waits a number of microseconds. It does not look after the USB devices, so a long wait should call poll() as it goes:

```
uint64_t end = cmm2->micros() + 500000;
while(cmm2->micros() < end) cmm2->poll();
```

Numbers

float and double are done in hardware, and the maths library (sin, sqrt, pow...) is there for both. printf() and scanf() handle floating point (%f, %g, %e). 64-bit integers (long long, int64_t) work, with division done in software.

What Is Not There

There is no operating system, so there are no processes (system(), fork(), exec()), threads, signals or sockets. No environment variables are set at the start (getenv() returns NULL), but setenv() sets one for the program to read: a library that is configured by them, as PDCurses is, can be given them so. A program runs alone, with the firmware's interrupts under it, until it ends.

The Services

The services are the program's way to the firmware: a table of functions, declared in Inc/cmm2img.h, whose address the firmware passes to the program at its start and crt0.c keeps in the variable cmm2. A program includes cmm2.h and calls them through it:

```
#include "cmm2.h"
...
cmm2->con_puts("Hello\r\n");
```

This chapter describes the general services: the console, the clock, the files and directories, the system and the FM synthesiser. The chapters after it describe the screen, the keyboard and mouse, sound and music, and the I/O pins. Appendix A lists them all in the table's order, with the version that added each.

Errors

A service never raises an MMBasic error: MMBasic's error handling would leave the program's stack and files in disorder. Each returns a status instead, negative for an error. For the file services it is the FatFs error code made negative:

Code	Meaning	Code	Meaning
-1	A disk error	-8	The file is there already
-3	The drive is not ready (no card or USB drive)	-9	Not an open file or directory
-4	No such file	-11	No such drive
-5	No such path	-13	No file system on the drive
-6	The name is not valid	-17	Not enough memory
-7	Access denied (read only, a directory, full)	-18	Too many open files

The Console

```
void con_putc(int c);
void con_puts(const char *s);
int  con_getc(void);
void con_screen(int on);                        /* version 4 */
```

con_putc and con_puts write to the console, as printf() does but without the C library's buffering and without turning a newline into a carriage return and line feed ("\r\n" is wanted for a new line).

con_getc returns the next key typed, from the serial console or a USB keyboard, or -1 if there is none: it never waits. Ordinary keys are their ASCII codes, and the others MMBasic's codes (the arrows are 128 to 131, the function keys F1 to F12 145 to 156 and so on: see Appendix I of the user manual). A USB keyboard's keys arrive only when poll() is called.

con_screen(0) sends the console's output to the serial port only, so that a program's messages are not drawn over its display (and do not scroll it). con_screen(1), the setting at the start, sends it to both.

The Clock

```
uint64_t micros(void);
void     delay_us(uint32_t us);
void     datetime(int t[6]);
```

micros returns the microseconds since the Colour Maximite 2 started, from a hardware timer. delay_us waits that many microseconds, without looking after the USB devices. datetime fills t with MMBasic's date and time: year, month, day, hour, minute and second.

Files

```
int     file_open(const char *name, int mode);
int     file_close(int h);
int32_t file_read(int h, void *buf, uint32_t n);
int32_t file_write(int h, const void *buf, uint32_t n);
int     file_seek(int h, uint32_t pos);
int32_t file_tell(int h);
int32_t file_size(int h);
int     file_sync(int h);
```

```
int     file_remove(const char *name);
int     file_rename(const char *from, const char *to);
int     file_stat(const char *name, cmm2_stat *st);    /* version 5 */
```

file_open opens a file and returns its handle, 0 to 7, or an error. The mode is made of these bits, combined with C's | operator:

Mode	Meaning
CMM2_O_READ	Read.
CMM2_O_WRITE	Write: the file must be there, unless CREATE or APPEND is added too.
CMM2_O_WRITE + CMM2_O_CREATE	The file is made, or emptied if it is there.
CMM2_O_WRITE + CMM2_O_APPEND	The file is made if it is not there, and written at its end.

READ and WRITE can be used together. A file opened to read, or to read and write without CREATE or APPEND (version 9), is given a map of its clusters, so that seeking in it does not have to read the card's allocation table again (Doom seeks to each part of its 28MB data file). A write or seek past the end of the file drops the map, as the file then grows.

file_read and file_write return the number of bytes read or written (fewer than asked at the end of a file, or when the drive is full) or an error. file_seek moves to a position from the start of the file. file_sync writes what is waiting to the card without closing the file. file_rename fails if a file of the new name is there (-8); the C library's rename() removes it first. Names are as described in the chapter The C Library.

file_stat gives a file's or directory's details:

```
typedef struct {
    uint32_t size;                              /* 0 for a directory */
    uint8_t  is_dir;
    uint8_t  month, day, hour, minute, second;  /* its last change */
    uint16_t year;
} cmm2_stat;
```

It returns 0, or an error (-4 if there is no such file).

Directories

```
int dir_make(const char *name);
int dir_open(const char *path);
int dir_read(int h, cmm2_dirent *e);
int dir_close(int h);
```

dir_make makes a directory. dir_open opens a directory for listing and returns its handle, 0 to 3. dir_read fills in the next entry and returns 1, or 0 when there are no more:

```
typedef struct {
    char     name[256];
    uint32_t size;
    uint8_t  is_dir;
} cmm2_dirent;
```

For example, to list the root of the SD card:

```
cmm2_dirent e;
int d = cmm2->dir_open("A:/");
if(d >= 0) {
    while(cmm2->dir_read(d, &e) == 1)
        printf("%s%s  %lu\n", e.name, e.is_dir ? "/" : "", (unsigned long)e.size);
    cmm2->dir_close(d);
}
```

The files and directories a program leaves open are closed when it ends.

The System

```
int      poll(void);
void     leave(int code);
uint32_t cpu_hz(void);
void    *alloc(uint32_t size);
void     release(void *p);
int      break_take(void);                      /* version 5 */
```

poll looks after the USB devices: the keyboard, the mouse, game controllers and the USB drive. It returns 1 once Ctrl-C has been pressed, as a request to end. It is quick when there is nothing to do, and a program should call it at least every 10 to 20ms: in its main loop, or once a frame.

leave ends the program from anywhere, with its exit code (see The End in the chapter How a Program Runs).

cpu_hz returns the processor's clock in Hz: normally 480000000 (400000000 with the earlier revision Y of the chip), and 504000000 under OPTION OVERCLOCK ON.

alloc takes a block of MMBasic's heap, outside the program's own memory, and returns it or NULL; release gives it back. Up to 64 blocks can be held, and they are freed when the program ends.

break_take returns 1 if Ctrl-C has been pressed since it was last called, and clears it, so that the program deals with Ctrl-C itself and the BASIC program that ran it carries on afterwards. poll() also reports a Ctrl-C until break_take takes it.

The FM Synthesiser

```
void *opl_new(uint32_t rate);
void  opl_write(void *chip, uint32_t reg, uint32_t value);
void  opl_generate(void *chip, int16_t *buf, uint32_t frames);
void  opl_free(void *chip);
```

These give a program the firmware's emulation of the Yamaha YMF262 (the OPL3 of the Sound Blaster cards, by ymfm), for a program that drives an OPL chip itself: a DOS game's AdLib music, or an emulator.

opl_new makes a chip, reset, that produces 16-bit stereo sound at rate samples a second (44100 for the sound stream), and returns it, or NULL if there is no room (its state, about 18KB, is a block of MMBasic's heap). opl_write writes a register (0 to 0x1FF: 0x100 and above are the OPL3's second bank) at once. opl_generate makes frames samples of interleaved stereo (left, right) into buf, running the chip. opl_free frees it.

The emulation is exact and takes time: about 5,000 processor cycles a sample with all 18 voices sounding, so 1,024 frames take about 10ms on a Generation 2. A program generates the sound in its own code and hands it to the sound stream (see the chapter Sound and Music). A real chip took about 23 microseconds to act on a register write: a program that writes a key off and a key on to the same voice together should generate at least one sample between them, or the note is not struck again. For MIDI music the music_play service is simpler: it plays a MIDI file on the firmware's own chip, as PLAY MIDIFILE does.

MMBasic's Statements and Expressions

```
int basic_cmd(const char *line);                          /* version 11 */
int basic_eval(const char *expr, char *out);              /* version 12 */
int basic_prep(const char *line, char *tokens, int size);
int basic_exec(char *tokens);
```

basic_cmd runs one MMBasic statement, a command with its arguments as text, as the interpreter runs it: it returns 0, or -4 with the message in io_error(), or -3 if the line is too long. It cannot call a SUB. basic_eval works out an expression, for what only the firmware knows, such as MM.INFO(SOUND), and gives its value as text in out, which has to be 256 bytes.

Tokenising a statement takes far longer than most statements take to run: 85 microseconds against 4 for PAGE WRITE. basic_prep does it once, with the character 1 in the line for each value to come, and basic_exec runs the tokens that it made, with the values written in as digits or a quoted string. A compiled MMBasic program uses these for the commands it hands to the firmware. Both use the interpreter, so a program should call them from its main code only.

The Screen

A program draws by writing pixels into display memory: the firmware sets up the video mode and gives the program the address and layout of the page that is shown. There are two ways to use it. The program can write straight into the page (Doom does), or it can keep its picture in its own memory and have it copied to the page at each frame, by the processor or the DMA2D (MicroPython does). When the program ends, MMBasic's video mode, font, colours and palette are put back.

Setting a Mode

```
int video_mode(int mode, int depth, cmm2_video *v);         /* version 2 */

typedef struct {
    uint8_t *fb;                /* the first pixel of the page shown */
    uint32_t width, height;     /* in pixels */
    uint32_t pitch;             /* bytes from a line's start to the next's */
    uint32_t bpp;               /* bits a pixel: 8 or 16 */
    uint32_t copies;            /* 1, or 2 in a mode whose lines are shown twice */
} cmm2_video;
```

video_mode sets MMBasic's MODE mode,depth, clears the page and fills in v. It returns 0, or -1 if the mode or depth is not valid on this computer (as MODE would refuse it: MODE 16 needs a Generation 2; or, on a Generation 1 that has lent the program its display memory, a mode whose page does not fit in the internal RAM: see The Display Memory Lent in the chapter How a Program Runs), or -2 if MMBasic has a frame buffer of another depth. The depth is 8 (256 colours from the palette) or, from version 6, 16 (RGB565: two bytes a pixel, 65,536 colours; not in modes 15 and 16), or, from version 17, 12 (ARGB4444: two bytes a pixel, 4,096 colours, in the modes that take 16). In depth 12 the screen is one layer whose pixels' alpha is not used, so a pixel is 0x0RGB, four bits each of red, green and blue, as an Amiga's colour registers hold them; v.bpp is 12 and the page is laid out as in depth 16. It is not MMBasic's 12-bit mode, with its two layers: MMBasic's drawing services and the console on the screen still colour as in depth 16, which shows wrongly but for black and white.

The modes are MMBasic's:

Mode	Size	Mode	Size	Notes
1	800 x 600	10	848 x 480	
2	640 x 400	11	1280 x 720	
3	320 x 200	12	960 x 540	lines shown twice: 3, 5, 6, 7, 12, 13, 17 and 19
4	480 x 432	13	400 x 300	
5	240 x 216	14	960 x 540	
6	256 x 240	15	1280 x 1024	8-bit only
7	320 x 240	16	1920 x 1080	Generation 2, 8-bit only
8	640 x 480	17	384 x 240	
9	1024 x 768	18	1024 x 600	
		19	640 x 512	mode 15 at half its pixel clock

Each pixel is a byte in depth 8 and two bytes (red in the top five bits, green in the middle six, blue in the bottom five) in depth 16. Line y starts at v.fb + y * v.pitch.

Modes whose lines are shown twice. Modes 3, 5, 6, 7, 12, 13, 17 and 19 are shown with each line repeated, and their page holds each line twice: v.copies is 2, v.pitch is twice a line's width, and line y must be written at v.fb + y * v.pitch and again just after it. vidtest.c and Doom do it this way:

```
static void line(int y, const uint8_t *px) {          /* depth 8 */
    uint8_t *p = v.fb + y * v.pitch;
    memcpy(p, px, v.width);
    if(v.copies == 2) memcpy(p + v.width, px, v.width);
}
```

A mode change makes the monitor find the new picture, which takes a second or two (the firmware cannot shorten it), so a program sets its mode once at the start.

The page is not held in the processor's data cache, so what the program writes is on the screen at once. It is slower to write than the program's own memory, a byte at a time most of all: build a line or a frame in the program's memory and copy it, with memcpy() or the DMA2D.

The Palette

```
int video_palette(int start, int count, const uint8_t *rgb);   /* version 2 */
```

In depth 8 each pixel is an index into a palette of 256 colours. video_palette sets count of them from start, from red, green and blue bytes (three for each colour), and returns 0, or -1 if the depth is not 8 or the range is wrong. It waits for the start of the next frame and loads the colours then, so the change is seen whole, and a palette changed every frame (for fades and colour cycling) costs nothing more. Doom loads its palette this way when it changes. MMBasic's palette is put back when the program ends.

Waiting for a Frame

```
void video_wait(void);                                          /* version 2 */
```

video_wait returns at the start of the display's next frame (or after 50ms, whatever happens). The frame rate depends on the mode (MODE 3 runs at 75 frames a second, MODE 18 at 60), and a program can time its frames with micros() to find it. Drawing just after video_wait, while the screen's first lines are being sent, gives a picture without tearing for a program that draws in the shown page.

A Display in the Program's Memory

Version 6 adds the services for a program that keeps its picture in its own memory and has it copied to the screen at each frame, as the Pico Computer 3's hdmi module does (MicroPython's display is built this way):

```
int  video_scaled(int mode, int depth, int scale, cmm2_video *v);
void frame_hook(void (*fn)(void));
int  dma2d_copy(const void *src, uint32_t src_pitch, void *dst, uint32_t dst_pitch,
                uint32_t width, uint32_t height);
int  dma2d_busy(void);
int  dma2d_copy_nc(const void *src, uint32_t src_pitch, void *dst, uint32_t dst_pitch,
                   uint32_t width, uint32_t height);                /* version 18 */
```

video_scaled is video_mode with each pixel shown scale times in each direction (scale is 1 or 2). With 2 the display is half the mode's width and height (MODE 18 at 2 gives 512 x 300), and its page holds each line twice, as in the modes whose lines are shown twice (v.copies is 2). It is refused in those modes, which are doubled already.

frame_hook asks for fn to be called at the start of each frame's blanking interval, the moment to copy a frame to the page without tearing, and frame_hook(NULL) stops it. fn is called in an interrupt and must follow the rules in Code Called From an Interrupt in the chapter Sound and Music: be quick, and call no services but dma2d_copy and dma2d_busy.

dma2d_copy has the processor's DMA2D copy a rectangle while the program goes on: width bytes from each of height lines, the source's lines src_pitch bytes apart and the destination's dst_pitch. Up to three copies wait their turn, each starting when the one before ends. It returns 0, or -1 if three are waiting already or a size, pitch or address is odd. It writes the source from the data cache to memory first, so the copy sees what the program last wrote; the program must not change the source until the copy is done. dma2d_busy returns how many copies are not yet done. A frame copied this way costs the processor very little. This example keeps a 320 x 240 picture in the program's memory and shows it in MODE 7, whose page holds each line twice, so it is copied twice, to the first and the second copy of each line:

```
static uint8_t frame[240][320];              /* the picture, depth 8 */
static cmm2_video v;

static void hook(void) {                     /* each frame, in an interrupt */
    cmm2->dma2d_copy(frame, 320, v.fb, v.pitch, 320, 240);
    cmm2->dma2d_copy(frame, 320, v.fb + v.width, v.pitch, 320, 240);
}

int main(int argc, char **argv) {
    if(cmm2->version < 6 || cmm2->video_mode(7, 8, &v) < 0) return 1;
    cmm2->frame_hook(hook);
    while(!cmm2->poll()) {
        draw(frame);                         /* the program's drawing */
    }
    cmm2->frame_hook(NULL);
```

```
    return 0;
}
```

dma2d_copy_nc, from version 18 of the services, is dma2d_copy without the source written from the data cache first. It is for a second copy of a source that a dma2d_copy has just written back, with nothing written to it since: writing a large source back takes time, and doing it twice is wasted. The Amiga emulator copies each batch of its rows to two lines of its page this way, the first copy by dma2d_copy and the second by dma2d_copy_nc.

A copy in progress while the program draws the next frame shows a mixture of the two: double buffering (two frames, drawing one while the other is copied) avoids that.

When the program ends its frame hook is removed and copies still waiting are dropped.

Drawing

Version 10 adds drawing by MMBasic's own routines, the ones its CLS, PIXEL, LINE, BOX, RBOX, CIRCLE, TRIANGLE, TEXT, FONT, COLOUR and BLIT READ and WRITE commands use, on the display video_mode set (or MMBasic's, if the program set none):

```
int gfx_cls(int colour);
int gfx_pixel(int x, int y, int colour);
int gfx_line(int x1, int y1, int x2, int y2, int width, int colour);
int gfx_box(int x, int y, int w, int h, int width, int colour, int fill);
int gfx_rbox(int x, int y, int w, int h, int radius, int colour, int fill);
int gfx_circle(int x, int y, int r, int width, float aspect, int colour, int fill);
int gfx_triangle(int x1, int y1, int x2, int y2, int x3, int y3, int colour, int
fill);
int gfx_text(int x, int y, const char *s, const char *just, int font, int scale, int
colour, int bg);
int gfx_font(int font, int scale, int *w, int *h);
int gfx_colour(int colour, int bg);
int gfx_read(int x, int y, int w, int h, uint8_t *bgr);
int gfx_write(int x, int y, int w, int h, const uint8_t *bgr);
int gfx_save(const char *name);
```

Colours are MMBasic's: RGB888 (0xRRGGBB, with MMBasic's transparency nibble in bits 24-27, as its RGB() gives them), made the mode's own, so the same calls draw in every mode and depth. A fill or background of -1 is none. The arguments are the commands': a box's w and h are its size, aspect 1 makes a circle round (2 an ellipse twice as wide), and gfx_text's just is TEXT's ("" for the default, "CM" centred on x and y...), its font 1-16 and scale 1-15, or 0 for the console's (FONT's). gfx_font sets the console's font, as FONT does (font 0 leaves it), and gives the size of its characters in pixels; gfx_colour sets the console's colours, as COLOUR does. gfx_read gives an area's pixels as 3 bytes each, blue, green and red (BLIT READ's buffer), and gfx_write puts them back. gfx_save saves the screen as a PNG, as SAVE PNG and Print Screen do (in about 0.4 seconds for 800 x 600), or as a BMP if the name ends in .BMP.

Each returns 0, or -4 when MMBasic's command would have given an error (a font that is not loaded, a justification it does not know, a folder that is not there), with the message in io_error(). Drawing this way is slower than writing the page directly, a call a shape, but the shapes, the fonts and the colours are MMBasic's own. A program can mix the two: the page video_mode gives is the one these draw on.

```
cmm2_video v;
if(cmm2->version < 10 || cmm2->video_mode(1, 8, &v) < 0) return 1;
cmm2->gfx_cls(0);
cmm2->gfx_box(20, 20, 160, 100, 3, 0x0FFFFFFF, 0x0FFF0000);   /* white, filled red */
cmm2->gfx_circle(400, 70, 50, 2, 1.0f, 0x0F00FF00, -1);       /* a green ring */
cmm2->gfx_text(400, 200, "Hello", "CM", 1, 3, 0x0FFFFFFF, -1);
cmm2->gfx_save("A:/hello.png");
```

The Keyboard and Mouse

A program reads the keyboard in two ways: as characters typed, for text, or as the keys held down at the moment, for games. The mouse is read as a position and buttons. A USB keyboard, mouse or game controller is looked after only when the program calls poll(), so a program that reads them calls it each time round its main loop.

Characters Typed

```
int con_getc(void);
```

con_getc returns the next character typed on the serial console or a USB keyboard, or -1 if there is none (it never waits). Letters, digits and symbols are their ASCII codes, Enter is 13, Backspace 8, Tab 9 and Escape 27. The other keys have MMBasic's codes:

Key	Code	Key	Code
Up arrow	128	Insert	132
Down arrow	129	Home	134
Left arrow	130	End	135
Right arrow	131	Page Up	136
Delete	127	Page Down	137
F1 to F12	145 to 156	Ctrl with a letter	1 to 26

The full list, with the keys combined with Shift, Ctrl and Alt, is in Appendix I of the user manual. getchar() and the other stdio input functions read the same keys, but wait for one and show it.

Keys Held Down

```
int keys_held(int keys[7]);                                    /* version 2 */
```

keys_held fills keys[0] to keys[5] with the keys held down on a USB keyboard at that moment, as MMBasic's KEYDOWN function gives them (0 for none), and keys[6] with the modifier keys held, as bits:

Bit	Key	Bit	Key
1	Left Alt	16	Right Alt
2	Left Ctrl	32	Right Ctrl
4	Left GUI (Windows)	64	Right GUI
8	Left Shift	128	Right Shift

It returns the number of keys held. The codes are as for con_getc: each is the character the key types, so Shift with a gives A (65) and Shift with 1 gives ! (33), and Ctrl with a letter gives its control code (Ctrl-D is 4). Up to six keys are reported at once, as the USB keyboard reports them.

A game reads the keys once a frame and compares them with the frame before, to find the keys pressed and released. In outline, Doom's layer works this way:

```
static int last[7];

static void keys(void) {
    int now[7], i, j;
    cmm2->poll();                                  /* the keyboard read */
    cmm2->keys_held(now);
    for(i = 0; i < 6; i++) {                       /* pressed: in now, not in last */
        if(now[i] == 0) continue;
        for(j = 0; j < 6 && last[j] != now[i]; j++) {}
        if(j == 6) pressed(now[i]);
    }
    for(i = 0; i < 6; i++) {                       /* released: in last, not in now */
        if(last[i] == 0) continue;
        for(j = 0; j < 6 && now[j] != last[i]; j++) {}
        if(j == 6) released(last[i]);
    }
    memcpy(last, now, sizeof(last));
}
```

The keys held come from a USB keyboard only: the serial console sends characters, not key presses, so a program that should also work from a terminal reads con_getc as well.

The Keyboard's Own Codes

```
int keys_raw(uint8_t report[8]);                    /* version 9 */
```

keys_raw gives the USB keyboard's last report as the keyboard sent it, before MMBasic's keyboard layout is applied. Each key is named by its place on the keyboard, not by the character it types, as a PC's scan codes name the keys. This is for a program that applies a layout of its own, or needs keys that type no character: the 386 PC emulator passes the keys to the PC this way, and DOS applies its own layout.

Byte	Holds
0	The modifier keys held, as bits in the USB order: 1 left Ctrl, 2 left Shift, 4 left Alt, 8 left GUI, 16 right Ctrl, 32 right Shift, 64 right Alt, 128 right GUI. Note that this order is not keys_held's.
1	0.
2 to 7	The USB HID usage codes of up to six keys held (4 for A, 5 for B, and so on to 0x65), or 0.

It returns the number of keys held. With no keyboard, the report is all zero.

The Mouse

```
int  mouse(int m[4]);                               /* version 4 */
void mouse_set(int x, int y);                       /* version 4 */
```

mouse reads the mouse that MMBasic is using (as MM.MOUSE chooses it: a USB mouse, a PS/2 mouse or an I2C mouse adapter). It returns 1 and fills m with its position in the screen's pixels, x then y, its buttons (1 left, 2 right, 4 middle, added together) and the count of its wheel; or 0, with m all zero, if there is no mouse. The firmware keeps the position within the screen, and video_mode centres it. mouse_set moves it. A program draws its own pointer.

```
int m[4];
if(cmm2->mouse(m) && (m[2] & 1)) clicked(m[0], m[1]);
```

Game Controllers

There is no service for USB game controllers yet. A program that wants one can read the keyboard instead, which every game can be played with.

USB Gamepads

```
int gamepad(int n, int *lx, int *ly);               /* version 13 */
int gamepad_axes(int n, int v[6]);                  /* version 16 */
```

gamepad gives the buttons of the pad in USB slot n, 1 upward, as a bitmap: the one DEVICE(GAMEPAD n, B) gives in BASIC, which the User Manual describes. If lx and ly are not NULL they are given the left stick, each 0 to 255 with 128 in the centre. A pad with no sticks reads as centred, and its d-pad is in the bitmap. It returns -1 if there is no pad in the slot. poll keeps it up to date, so a program calls poll regularly. A DualShock 3 sends nothing until its PS button is pressed.

gamepad_axes, from version 16 of the services, gives the same buttons, and in v all of the pad's axes as DEVICE(GAMEPAD n, ...) gives them: v[0] and v[1] the left stick (LX, LY), v[2] and v[3] the right stick (RX, RY), each 0 to 255 with 128 in the centre, and v[4] and v[5] the analogue L2 and R2 triggers (L, R), 0 to 255. A pad with no sticks reads as centred, and one without analogue triggers as 0. apps/atari800 makes a single pad's right stick the Atari's second joystick with it, for twin-stick games, and uses gamepad instead on older firmware:

```
if(cmm2->version >= 16) b = cmm2->gamepad_axes(slot, v);
else b = cmm2->gamepad(slot, &v[0], &v[1]);
```

apps/nes/nes_cmm2.c reads the first two pads it finds in this way, and turns them into a NES's buttons.

Sound and Music

A program makes sound by filling buffers: the firmware plays a stream of 16-bit stereo samples on the audio output and calls a function of the program's whenever it needs another buffer. On top of that the firmware can play MIDI music on its FM synthesiser, mixed into the same stream. PLAY VOLUME's setting applies to both.

The Sound Stream

```
int  audio_start(void (*fill)(int16_t *buf, uint32_t frames));    /* version 3 */
void audio_stop(void);
void audio_lock(void);
void audio_unlock(void);
```

audio_start starts the stream and returns its rate in samples a second (about 44100: the exact rate the timer makes), or -2 if MMBasic is playing a sound, or -3 if there is no memory for the buffers. From then on the firmware calls fill each time a buffer is wanted, with buf to fill with frames pairs of samples (left, right, left, right...). A buffer is 1,024 frames, 23ms of sound, and there are two: while one plays the other is filled.

audio_stop stops it, and it is stopped when the program ends.

This program plays a tone whose pitch follows the mouse:

```
#include <math.h>
#include "cmm2.h"

static volatile float step;                            /* the phase step of a sample */
static float phase;

static void fill(int16_t *buf, uint32_t frames) {
    uint32_t i;
    for(i = 0; i < frames; i++) {
        int16_t s = (int16_t)(8000 * sinf(phase));
        buf[2 * i] = buf[2 * i + 1] = s;
        phase += step;
        if(phase > 6.2831853f) phase -= 6.2831853f;
    }
}

int main(int argc, char **argv) {
    int m[4], rate;
    if(cmm2->version < 4) return 1;
    rate = cmm2->audio_start(fill);
    if(rate < 0) return 2;
    while(!cmm2->poll()) {
        float hz = cmm2->mouse(m) ? 200 + m[0] : 440;
        step = 6.2831853f * hz / rate;               /* one float: no lock needed */
        cmm2->delay_us(10000);
    }
    cmm2->audio_stop();
    return 0;
}
```

Code Called From an Interrupt

From version 18 of the services (7.00.00RC5) the stream goes to the DACs by DMA, with no interrupt for each sample: its cost to the program is then the fill and the music alone. fill (and a frame hook, in the chapter The Screen) is called from an interrupt, PendSV, at the lowest priority of all the firmware's interrupts, while the program's own code is stopped. So it must follow some rules:

- **Be quick.** Its time comes off the program's: a fill that takes 5ms of each 23ms buffer takes over a fifth of the processor. It must always finish well within 23ms, or the sound breaks up.
- **Keep its local variables small.** It runs on MMBasic's stack, not the program's, and that has a few kilobytes to spare at most. Large tables belong in static memory.
- **Call no services** (but dma2d_copy, dma2d_copy_nc and dma2d_busy from a frame hook), and no malloc(), free(), printf() or other C library function that keeps state. Calculation (string.h, math.h) is fine.
- **Share data with care.** A variable that both fill and the program change should be volatile. When the program changes more than one thing that fill reads (a voice's start, length and step, say), it does so between audio_lock() and audio_unlock(), which hold fill off: briefly, as the buffers run on meanwhile.

Doom's sound effects are mixed this way: its fill adds up sixteen voices of 8-bit samples, and its sound functions set up a voice inside audio_lock() and audio_unlock().

Music

```
/* version 3 */
int  music_play(const void *midi, uint32_t size, int loop, const uint8_t *bank);
void music_stop(void);
void music_volume(int volume);
void music_pause(int paused);
int  music_playing(void);
void music_rate(int divisor);
```

music_play plays a Standard MIDI File (types 0 and 1), held in the program's memory, on the firmware's FM synthesiser, as PLAY MIDIFILE does: the music is added to the stream after fill has made each buffer, so audio_start must be called first (a program with no sound of its own gives it a fill that makes silence). The file must stay in memory while it plays. loop plays it again from the start each time it ends. bank is a GENMIDI instrument bank, the format that Doom and its relatives use: 175 instruments of 36 bytes (the 128 General MIDI instruments, then the drums of notes 35 to 81), the bytes after a GENMIDI lump's 8-byte header "#OPL_II#". NULL uses the firmware's bank, which is Freedoom's. It returns 0, -1 if the data is not a MIDI file, -2 if its timing is in SMPTE frames, or -3 if there is no memory for the synthesiser.

music_stop stops it. music_volume sets its level, 0 to 127 (127 at the start). music_pause(1) pauses it and music_pause(0) resumes it. music_playing returns 0 once a file not looped has ended.

The synthesiser takes a large share of the processor: on a Generation 2, from about 20% with no notes sounding to about 54% with all 18 voices (a Generation 1 at 400MHz needs a fifth more). music_rate(n) makes it produce its sound at 1/n of its full rate of 49,716 samples a second (n is 1 to 4), which cuts its share without changing the pitch or the timing, at the cost of some treble. Doom asks for half the rate, which takes its music from 51% to 37% of the processor; the full rate is the default.

This program plays a MIDI file once, named after RUN's comma, with the firmware's instruments:

```
#include <stdio.h>
#include <string.h>
#include "cmm2.h"

static uint8_t song[256 * 1024];

static void silence(int16_t *buf, uint32_t frames) { memset(buf, 0, frames * 4); }

int main(int argc, char **argv) {
    FILE *f = fopen(argc > 1 ? argv[1] : "music.mid", "rb");
    uint32_t n;
    if(f == NULL || cmm2->version < 3) return 1;
    n = fread(song, 1, sizeof(song), f);
    fclose(f);
    if(cmm2->audio_start(silence) < 0) return 2;
    if(cmm2->music_play(song, n, 0, NULL) < 0) return 3;
    while(cmm2->music_playing() && !cmm2->poll()) cmm2->delay_us(10000);
    return 0;                          /* the sound stops as the program ends */
}
```

The FM Synthesiser Directly

For a program that drives an OPL chip register by register (a DOS game's AdLib driver, an emulator), the opl_ services give it chips of its own: see The FM Synthesiser in the chapter The Services. It makes their sound in its main code into a buffer of its own, and its fill copies from that buffer.

The I/O Pins

The services of version 7 give a program the pins of the rear I/O connector: digital inputs and outputs, the analog inputs, PWM, I2C, SPI and the COM ports. They name the pins by the PicoMite's GP numbers, as MMBasic does, so that code written for a PicoMite (MicroPython's machine module, for one) carries over with its wiring. The firmware's own drivers do the work, so the pins behave as they do for a BASIC program.

The Pins

Each GP number is the pin of the connector that has that function:

GP	Pin	Function	GP	Pin	Function
GP0	8	COM1 TX	GP14	27	I2C2 SDA
GP1	10	COM1 RX	GP15	28	I2C2 SCL
GP2	12	PWM 1A	GP16	3	I2C SDA
GP3	22	PWM 1B	GP17	5	I2C SCL
GP4	36	PWM 2A	GP18	23	SPI clock
GP5	31	PWM 2B	GP19	19	SPI MOSI
GP6	7	General I/O	GP20	21	SPI MISO
GP7	13	General I/O	GP21	33	General I/O
GP8	15	General I/O	GP22	32	General I/O
GP9	24	General I/O	GP26	26	Analog input
GP10	40	SPI2 clock	GP27	29	Analog input
GP11	38	SPI2 MOSI	GP28	37	Analog input
GP12	35	SPI2 MISO	GP36	16	COM2 TX
GP13	18	General I/O	GP37	11	COM2 RX

Any of them can be a digital input or output when its other function is not in use. Pin 40 (GP10) is also the reset link: connected to ground (pin 39) at power up it resets every option and erases the program in program memory, so anything wired to it must not look like a short to ground at power up (see the user manual).

Errors

Each I/O service returns 0 (or a value) or an error:

Error	Meaning
-1	No such GP number, bus or port.
-2	The pin is in use by a COM port, SPI, I2C or another function.
-3	Not possible: that mode on that pin, or a bus or port that is not open.
-4	The firmware's driver reported an error: io_error() returns its message.

```
const char *io_error(void);
```

Digital and Analog Pins

```
int  pin_setup(int gp, int mode, int pull);
int  pin_read(int gp);
void pin_write(int gp, int v);
int  adc_read(int gp, uint32_t *uv);
```

pin_setup sets a pin's mode: CMM2_PIN_OFF (not configured), CMM2_PIN_IN, CMM2_PIN_OUT, CMM2_PIN_OD (an open drain output) or CMM2_PIN_ANALOG. For an input, pull is CMM2_PULL_NONE, CMM2_PULL_UP or CMM2_PULL_DOWN. pin_read returns its level, 0 or 1 (an output's too), and pin_write sets an output's level.

adc_read reads an analog input: it returns 0 to 65535 for 0V to the supply voltage, and puts the voltage in microvolts in uv (if uv is not NULL), from 3.3V or OPTION VCC's setting.

```
cmm2->pin_setup(21, CMM2_PIN_OUT, CMM2_PULL_NONE);      /* GP21: pin 33 */
cmm2->pin_setup(26, CMM2_PIN_ANALOG, CMM2_PULL_NONE);   /* GP26: pin 26 */
for(;;) {
```

```
    uint32_t uv;
    cmm2->adc_read(26, &uv);
    cmm2->pin_write(21, uv > 1650000);                  /* high above half of 3.3V */
    if(cmm2->poll()) break;
}
```

PWM

```
int pwm_set(int gp, uint32_t freq, uint32_t duty);
int pwm_stop(int gp);
```

pwm_set starts or changes a PWM output, on GP2 to GP5: freq is its frequency in Hz (1 to 20,000,000) and duty its duty cycle, 0 to 65535 for 0% to 100%. GP2 and GP3 are the two outputs of PWM channel 1 and share its frequency, as GP4 and GP5 share channel 2's: setting one's frequency changes the other's, and the other keeps its duty cycle. pwm_stop stops an output.

I2C

```
int i2c_open(int bus, int khz, int timeout_ms);
int i2c_close(int bus);
int i2c_xfer(int bus, int addr, const uint8_t *w, uint32_t wn,
             uint8_t *r, uint32_t rn, int stop);
```

Bus 0 is I2C (GP16 and GP17) and bus 1 is I2C2 (GP14 and GP15). i2c_open opens one at 100, 400 or 1000 kHz, with a timeout of 0 (none) or 100ms or more. i2c_xfer writes wn bytes from w to the device at the 7-bit address addr, then reads rn bytes into r, with a repeated start between them, and releases the bus at the end if stop is not zero (0 keeps it, for a repeated start in the next transfer). With wn and rn both 0 it sends the address alone, to see whether a device answers. It returns 0, 1 if the device did not acknowledge, 2 if it timed out (as MM.I2C does in BASIC), or an error.

```
uint8_t reg = 0, secs;
cmm2->i2c_open(0, 100, 100);
/* a DS3231's seconds register */
if(cmm2->i2c_xfer(0, 0x68, &reg, 1, &secs, 1, 1) == 0)
    printf("seconds: %02x\n", secs);
cmm2->i2c_close(0);
```

SPI

```
int spi_open(int bus, uint32_t hz, int mode);
int spi_close(int bus);
int spi_xfer(int bus, const uint8_t *tx, uint8_t *rx, uint32_t n);
```

Bus 0 is SPI (GP18 to GP20) and bus 1 is SPI2 (GP10 to GP12). spi_open opens one at hz with the SPI mode 0 to 3, eight bits a transfer. spi_xfer sends n bytes from tx while receiving n into rx; tx NULL sends zeros and rx NULL throws away what comes in. A chip select is an ordinary output, set with pin_write around the transfer.

The COM Ports

```
int com_open(int com, uint32_t baud, int bits, int parity, int stop, uint32_t rxbuf);
int com_close(int com);
int com_write(int com, const uint8_t *buf, uint32_t n);
int com_read(int com, uint8_t *buf, uint32_t n);
int com_avail(int com);
int com_connected(int com);
```

com is 1 (COM1: GP0 and GP1), 2 (COM2: GP36 and GP37) or, from version 14 of the services, 4 to 7 (COM4 to COM7: USB serial devices, such as a PicoMite plugged into a USB port). A USB device moves its data when poll is called, and com_write returns fewer bytes than it was given if the device is unplugged, or stops reading and Ctrl-C is pressed. com_open opens it as MMBasic's OPEN "COMn:" does: bits 7 or 8, parity 0 (none), 1 (even) or 2 (odd), stop 1 or 2, and a receive buffer of rxbuf bytes (0 for MMBasic's default). com_write queues bytes to send (it waits while the queue is full) and returns how many. com_read takes up to n of the bytes received and returns how many it took, 0 if there are none: it does not wait. com_avail returns the number of bytes waiting.

com_connected, from version 15 of the services, says whether a port's device is there, open or not: 1 if it is, 0 if a USB serial device is unplugged or restarting, as a PicoMite does after many OPTION commands, and -1 if com is not one of those above. COM1 and COM2 are always 1. An open port stays open while its device is away, and when it comes back the firmware sets the line up again as it was, so a program has nothing to do but tell its user.

At the End

Whatever a program set up is undone when it ends: its pins are returned to not configured, its PWM outputs stopped, and its I2C and SPI buses and COM ports closed.

Performance

The STM32H743 is fast, and a loadable program has all of it: no interpreter, no operating system, and the firmware's interrupts take only a little of its time when nothing else is going on. A few things about the way it is built decide how fast a program runs.

Code and Data in the SDRAM

A program runs from the SDRAM, as its data is. The SDRAM is much slower than the processor, so the speed comes from the processor's two caches: 16KB for instructions and 16KB for data. A loop that fits in the instruction cache and works on data that fits in the data cache runs at the processor's full speed; code that jumps all over a large program, or data read once from a large area, runs at the SDRAM's.

Some consequences:

- **Measure, don't guess.** The same code can run at a different speed when a change elsewhere moves it in memory. When MicroPython was ported, adding a module that its benchmark never called made the benchmark 31% slower, by moving its interpreter's loop.
- **-O2 is usually best.** -O3 and loop unrolling make the code larger and often slower for the cache; -Os makes it smaller but slower in the loops that matter. Try --cflag=-O3 on a single hot file rather than on everything.
- **Keep hot data together.** Arrays of structures that are walked in order use the cache well; linked lists and tables of pointers scattered across a large heap do not.
- **The display memory is not cached**, so build a picture in the program's memory and copy it to the page (with memcpy() or the DMA2D) rather than drawing into the page a pixel at a time.

The DTCM is the firmware's. On a Generation 2 a program can put its hottest code and data in the AXI RAM and the ITCM: see Code in the Fast Memories below.

Code in the Fast Memories

On a Generation 2 whose display page is in the SDRAM, the processor's 512KB of AXI RAM is free, and the 24KB of ITCM from 0xA000 to 0xFFFF is the program's from version 18 of the services. build.py's --axi NAME links a source's code, constants, data and zeroed data to run in the AXI RAM, and --itcm NAME:F1,F2 links the functions named to run in the ITCM (or all of the source's code, with no names). The image carries them, and the program copies them into place by calling cmm2_axi_load() once video_mode has set its display (the modes clear the AXI RAM), and cmm2_axi_unload() before it ends:

```
if(cmm2->video_mode(1, 12, &v) < 0 || cmm2_axi_load()) return 1;
...
cmm2_axi_unload();
```

cmm2_axi_load() returns -1 if the AXI RAM is not free (the display's page is there). The AXI RAM runs through the caches, as the SDRAM does, but a cache miss costs far less; the ITCM needs no cache, and leaves the instruction cache to the rest. A call between the ITCM and the AXI RAM or the SDRAM, more than 16MB apart, goes through a short veneer the linker adds, so keep a function and the ones it calls most in the same memory. The Amiga emulator keeps its chips' code in the ITCM and Musashi's 68000, with its 128KB of opcode tables, in the AXI RAM: from the SDRAM alone it missed the caches at nearly every 68000 instruction and could not keep up.

Floating Point

The floating point unit does single and double precision in hardware: an addition or a multiplication takes a cycle or two, and a division or a square root about 14 cycles in single precision and 30 in double. So float and double cost about the same, except for division, square roots and the functions of math.h, where float is quicker (sinf() and the other float versions are quicker than sin()).

The Interrupts

While a program runs the firmware still does its work in interrupts: the USB host, the console's serial port, the video's frame interrupt, and the sound while it plays. Without sound this takes very little. Sound takes more: the program's fill and the FM synthesiser's music take what they take. Before version 18 of the services a sample interrupt also ran 44,100 times a second, about 4% of the processor; from version 18 the samples go to the DACs by DMA. Doom's timed demo runs at 97 frames a second silent and at 51 with its music and effects, the music at half its full rate.

Some Figures

On a Generation 2 at 480MHz:

What	Speed
Doom, timed demo 1, silent	97 frames a second
Doom, with music at half rate and effects	51 frames a second
MicroPython, the solar eclipse benchmark	4.4 seconds (8.8 on a Pico Computer 3; MMBasic 6.1 interpreted, 1.7 with OPTION COMPILE ON)
The FM synthesiser, 18 voices sounding	54% of the processor

A Generation 1 at 400MHz runs a little slower than its clock alone would suggest: Doom's demo runs at 78 frames a second silent and 35 with sound.

Debugging

There is no debugger on the Colour Maximite 2 itself, but a program has a console, and the files that build.py leaves beside the program file describe it exactly.

printf

The console is the first tool: printf() to the serial port (with con_screen(0), so that the messages stay off the program's display), read in a terminal emulator on the PC. stdout is buffered until a newline; fflush(stdout) or a message ending in a newline shows it at once. For timing, micros() measures any stretch of code to the microsecond.

A program that might crash can write its progress to a file instead, with fflush() and the file closed and opened again (or file_sync), so that what was written survives the restart.

When a Program Faults

A fault (a bad pointer, a stack overflow into nothing, a jump to a wrong address) restarts the Colour Maximite 2. To find out where:

- Narrow it down with messages: print as the program goes, and see which was the last.
- Look the addresses up. On a Generation 2 the program runs at the address it was linked for, so an address the program prints (say, a function pointer, or the return address from `__builtin_return_address(0)`) can be looked up in out/name.map, or with addr2line:

  ```
  arm-none-eabi-addr2line -f -e out/name.elf 0xD1801234
  ```

On a Generation 1 the program is moved: subtract cmm2_start->base and add 0xD1800000 first.

- With an ST-Link connected to the processor's SWD pins, STM32CubeProgrammer can read the processor's registers from a program that has stopped or is stuck in a loop, without resetting it:

  ```
  STM32_Programmer_CLI -c port=SWD mode=HOTPLUG -coreReg
  ```

The PC and LR registers it shows are looked up with addr2line as above.

Memory

Running out of memory shows as malloc() returning NULL (check it), or as a stack overflow, which corrupts the top of the heap without any message. Print cmm2_start->heap_size at the start to see what the program has, and give a program with deep recursion or large local arrays more stack with --stack.

The map file shows what takes the space in the image: the sizes of the sections of each object file, and of each function with -ffunction-sections, which build.py uses.

Testing the Moves

A program that runs on a Generation 2 but faults on a Generation 1 may hold an address that was not relocated. Build it with --base 0x60000000 and run it on a Generation 2: it is then moved too, and a missed address points at memory that is not there and faults at once, near the code that used it.

Running Again Quickly

Building, copying and running can be one step. On the PC the program is sent with ZMODEM (tools/zmodem_pc.py), and RUN typed over the serial port: the repository's tools/cmm2serial.py sends a line to the Colour Maximite 2 and prints the reply. Note that it sends Ctrl-C first, to get to the command prompt, which stops a program that is running.

Porting Existing Software

Much good software is written in portable C or C++ with a thin layer for each machine it runs on. Porting it to the Colour Maximite 2 is mostly a matter of writing that layer on the services. The three programs included with the firmware were ported this way, and each shows a different approach.

The Approach

1. **Find the platform layer.** Look for the files that open a window, read keys, play sound and read files: often one directory (backends, platform, port) or a few files with the platform's name in them. Everything else should compile unchanged. 2. **Build without it first.** Compile the portable part with build.py and stubs for the platform functions, and fix what the compiler finds: missing headers (POSIX ones such as unistd.h and dirent.h are not there), sizes of int and long (both 32 bits here), and functions the C library does not have. 3. **Write the layer** on the services: video_mode (or video_scaled and a frame hook) for the display, keys_held and con_getc for input, the sound stream for audio, and the C library's files. 4. **Fit the memory.** Size the program's buffers from cmm2_start->heap_size, and give a program with its own allocator one block with cmm2_heap_take(). 5. **Keep the original untouched.** Fetch the project's source at a known commit and keep the port's own files separate, so that the port can be rebuilt, updated and checked against the original. A GPL program's source must be given with the program, and this makes that easy: the original's location and commit, and the port's files.

Doom

Doom was written to be ported: its game code calls a small set of platform functions. doomgeneric reduces them to six (DG_Init, DG_DrawFrame, DG_SleepMs, DG_GetTicksMs, DG_GetKey and DG_SetWindowTitle), and the port is one file, apps/doom/doomgeneric_cmm2.c, of about 350 lines:

- **The picture.** DG_Init sets MODE 3,8 (320 x 200, 256 colours), Doom's own resolution and colour depth. DG_DrawFrame copies Doom's frame into the page, each line twice as the mode needs, and loads Doom's palette with video_palette when it changes.
- **The keys.** keys_held is read each frame and its changes are turned into Doom's key presses and releases, with the codes mapped to Doom's (Ctrl fires, Space opens, the arrows move).
- **The time.** DG_GetTicksMs is micros() divided by 1000, and DG_SleepMs waits while calling poll().
- **The sound effects.** Doom's sound functions are replaced by a mixer of sixteen voices in the stream's fill.
- **The music.** Doom's MUS music is turned into MIDI (by doomgeneric's mus2mid.c) and played by music_play with the game data's own instrument bank, as a Sound Blaster played it, at half the synthesiser's rate.
- **The memory.** Doom's zone, the memory it manages itself, is sized from the program's heap, up to doomgeneric's 6MB. Doom is built with --vram, so a Generation 1 lends it the display memory and it has the 6MB there too.

apps/doom/make.py fetches doomgeneric at a fixed commit, builds its files unchanged (with mus2mid.c, which its own make file leaves out) and the CMM2 layer, and links doom.app.

ScummVM

ScummVM is a large C++ program (531 source files in this port), and frank-quest had already ported it to a small machine, the RP2350, with its display, sound and file drivers in a handful of files. The CMM2 port replaces those files and nothing else:

- apps/scummvm/make.py reads frank-quest's own CMakeLists.txt for its lists of source files, so the build follows the original's.
- The directory overlay/ holds the replacements, at the same paths as the files they replace: the start-up and display driver (main.c, MODE 7,8 at 320 x 240), the input (keys_held and the mouse), the sound mixer's connection to the stream, the files (the file services), the memory allocator (the heap but 512KB, taken with cmm2_heap_take and managed by dlmalloc), and the list of games, with one more game and a game started by the name given after RUN's comma.
- The directory stub/ holds the few Pico SDK headers that the engines include, with just the declarations they use.

The program is 3MB and needs about 4MB of heap besides, more than a Generation 1 has borrowing MMBasic's heap alone, so it is built with --vram: a Generation 1 lends it the display memory too, and its MODE 7 display is one that a lent program can use.

MicroPython

MicroPython has its own build system, so it is built by its own make files with the tools' files (see Using Another Build System in the chapter Building Programs). It is a port in the usual MicroPython sense, a directory ports/cmm2 in MicroPython's tree, and much of it was taken from the Pico Computer 3's port so that the same Python programs run:

- **The REPL** reads con_getc, turning MMBasic's key codes back into the escape sequences that MicroPython's line editor expects, and writes with con_puts. Ctrl-C is taken with break_take and raised as KeyboardInterrupt.
- **The file system** is a MicroPython VFS on the file services: the SD card at / and the USB drive at /usb.
- **The display** keeps the Pico Computer 3's frame buffer in the program's memory and copies it to the page in a frame hook, by the DMA2D (or by the processor where a mode's pixels must be converted), so the hdmi module's modes and drawing work unchanged.
- **Sound** is the Pico Computer 3's I2S driver turned into a fill function, so its audio module plays WAV, MP3, FLAC and MOD files.
- **The machine module's Pin, ADC, PWM, I2C, SPI and UART** are written on the I/O services, with the PicoMite's GP numbers.
- **Memory**: the garbage-collected heap is taken with cmm2_heap_take, from what is left after the C library's share, and SQLite's memory pool is sized from the heap too (2MB on a Generation 2, 512KB on a Generation 1).

Things That Catch Ports

- **Case-insensitive file names on Windows**: a project's header can be found in place of another of the same name (ScummVM's audio.h and the firmware's Audio.h).
- **Assertions**: some code asserts things that are only true on the systems it was tested on. ScummVM's port is built with NDEBUG, as frank-quest's release builds are.
- **Large local arrays** overflow the 64KB stack without warning: give the program more with --stack.
- **Busy loops** that wait for something without calling poll() leave the keyboard and mouse dead.
- **Exceptions**: C++ code that relies on them needs changing, as they are not supported.

Appendix A – The Services

The services in the order of the table (struct cmm2_services, in Inc/cmm2img.h), with the version of the firmware's services that added each and the chapter that describes it. The table starts with two numbers: version, the firmware's version of the services (9 in 7.00.00RC1), and size, the table's size in bytes.

Ver	Service	Ver	Service
1	`void con_putc(int c)`	1	`void *alloc(uint32_t size)`
1	`void con_puts(const char *s)`	1	`void release(void *p)`
1	`int con_getc(void)`	1	`void *opl_new(uint32_t rate)`
1	`uint64_t micros(void)`	1	`void opl_write(chip, reg, value)`
1	`void delay_us(uint32_t us)`	1	`void opl_generate(chip, buf, frames)`
1	`void datetime(int t[6])`	1	`void opl_free(void *chip)`
1	`int file_open(name, mode)`	2	`int video_mode(mode, depth, v)`
1	`int file_close(int h)`	2	`int video_palette(start, count, rgb)`
1	`int32_t file_read(h, buf, n)`	2	`void video_wait(void)`
1	`int32_t file_write(h, buf, n)`	2	`int keys_held(int keys[7])`
1	`int file_seek(int h, uint32_t pos)`	3	`int audio_start(fill)`
1	`int32_t file_tell(int h)`	3	`void audio_stop(void)`
1	`int32_t file_size(int h)`	3	`void audio_lock(void)`
1	`int file_sync(int h)`	3	`void audio_unlock(void)`
1	`int file_remove(const char *name)`	3	`int music_play(midi, size, loop, bank)`
1	`int file_rename(from, to)`	3	`void music_stop(void)`
1	`int dir_make(const char *name)`	3	`void music_volume(int volume)`
1	`int dir_open(const char *path)`	3	`void music_pause(int paused)`
1	`int dir_read(int h, cmm2_dirent *e)`	3	`int music_playing(void)`
1	`int dir_close(int h)`	3	`void music_rate(int divisor)`
1	`int poll(void)`	4	`int mouse(int m[4])`
1	`void leave(int code)`	4	`void mouse_set(int x, int y)`
1	`uint32_t cpu_hz(void)`	4	`void con_screen(int on)`
Ver	**Service**	**Ver**	**Service**
5	`int break_take(void)`	7	`int i2c_open(bus, khz, timeout_ms)`
5	`int file_stat(name, cmm2_stat *st)`	7	`int i2c_close(int bus)`

6	int video_scaled(mode, depth, scale, v)	7	int i2c_xfer(bus, addr, w, wn, r, rn, stop)
6	void frame_hook(void (*fn)(void))	7	int spi_open(bus, hz, mode)
6	int dma2d_copy(src, sp, dst, dp, w, h)	7	int spi_close(int bus)
6	int dma2d_busy(void)	7	int spi_xfer(bus, tx, rx, n)
7	int pin_setup(gp, mode, pull)	7	int com_open(com, baud, bits, parity, stop, rxbuf)
7	int pin_read(int gp)	7	int com_close(int com)
7	void pin_write(int gp, int v)	7	int com_write(com, buf, n)
7	int adc_read(int gp, uint32_t *uv)	7	int com_read(com, buf, n)
7	int pwm_set(gp, freq, duty)	7	int com_avail(int com)
7	int pwm_stop(int gp)	7	const char *io_error(void)
9	int keys_raw(uint8_t report[8])	10	int gfx_text(x, y, s, just, font, scale, c, bg)
10	int gfx_cls(int colour)	10	int gfx_font(font, scale, *w, *h)
10	int gfx_pixel(x, y, colour)	10	int gfx_colour(colour, bg)
10	int gfx_line(x1, y1, x2, y2, width, colour)	10	int gfx_read(x, y, w, h, uint8_t *bgr)
10	int gfx_box(x, y, w, h, width, colour, fill)	10	int gfx_write(x, y, w, h, bgr)
10	int gfx_rbox(x, y, w, h, radius, colour, fill)	10	int gfx_save(const char *name)
10	int gfx_circle(x, y, r, width, aspect, c, fill)		
10	int gfx_triangle(x1, y1, x2, y2, x3, y3, c, fill)	10	int dir_change(const char *path)
10	int dir_current(char *buf, uint32_t size)	11	void *mem_copy(d, s, n)
11	void *mem_fill(d, c, n)	11	void *mem_move(d, s, n)
11	const struct cmm2_libm *libm	11	int basic_cmd(const char *line)
11	int gfx_target(cmm2_target *t)	11	int font_size(font, *w, *h)
11	int font_define(font, const uint8_t *data)	12	int basic_eval(expr, char *out)
12	int sound_state(void)	12	int basic_prep(line, char *tokens, size)
12	int basic_exec(char *tokens)	12	int image_resize(const long long *a)
13	int gamepad(n, *lx, *ly)	15	int com_connected(int com)
16	int gamepad_axes(n, int v[6])		

18	`int dma2d_copy_nc(src, sp, dst, dp, w, h)`		

The console, the clock, files, directories, the system and the FM synthesiser are in the chapter The Services; the screen and drawing in The Screen; the keys and mouse in The Keyboard and Mouse; audio and music in Sound and Music; and the I/O services in The I/O Pins. Inc/cmm2img.h has the full declarations, with a comment on each.

Appendix B – The Program File

A program file (.APP) is made by mkimage.py and read by the firmware's loader (Src/Loader.c). It has three parts: a 64-byte header, the image, and the relocations. Every number is little-endian.

The Header

Offset	Size	Field	Meaning
0	8	magic	"CMM2IMG" and a zero byte.
8	2	hdr_version	The header's version: 1.
10	2	api_version	The version of the services the program needs (mkimage's --api).
12	1	kind	1: an application, which runs and returns to MMBasic.
13	1	flags	Bit 0: the program needs a Generation 2 (--g2). Bit 1: a Generation 1 lends it the display memory (--vram).
14	2	reserved	0.
16	4	link_base	The address the program was linked at, a multiple of 64KB (0xD1800000).
20	4	image_size	The bytes of the image, which follow the header.
24	4	bss_size	The bytes of .bss, cleared to zero after the image.
28	4	entry	The entry point's offset in the image, plus 1 (a Thumb address).
32	4	stack_size	The stack's size, or 0 for 64KB.
36	4	heap_min	The least heap the program can run with.
40	4	reloc_count	The number of relocation words after the image.
44	4	crc32	The CRC-32 (zlib's) of the image and the relocation words.
48	16	name	The program's name, padded with zero bytes.

The Image

image_size bytes: the program as linked, from link_base: its code, constants, the tables of constructors and destructors, and its initialised data, with any gaps between them filled with zeros. The loader reads it to its place, then clears the .bss after it.

The Relocations

reloc_count 32-bit words follow the image. Each is the offset in the image of a place that holds an absolute address of the program, in its bottom 28 bits, with its kind in the top 4:

Kind	Meaning
0	A 32-bit word holding an address: the difference is added to it.
1	A Thumb-2 MOVT instruction, which loads the top half of an address: the top half of the difference is added to its 16-bit immediate.

The loader only moves a program by a multiple of 64KB, so the bottom half of an address (loaded by a MOVW) never changes and has no relocation. When the program is loaded at link_base (a Generation 2) the relocations are not used. On a Generation 1 the loader reads them into the memory just after the .bss, before the program's heap is set up there, so they need no memory of their own.

The loader checks the CRC before moving anything, and refuses a file whose relocation points outside the image or has an unknown kind ("The program file is damaged").

Appendix C – Licences

The Tools

The files that are built into every program, tools/sdk/crt0.c, syscalls.c, cmm2.h and app.ld, and the header Inc/cmm2img.h, are under the MIT licence, as are the examples (hello.c, vidtest.c, nopoll.c, oplbench.c and oplplay.c):

Copyright (c) 2026 Peter Mather.

Permission is hereby granted, free of charge, to any person obtaining a copy of this file, to deal in it without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of it, and to permit persons to whom it is furnished to do so, subject to the following condition: the above copyright notice and this permission notice shall be included in all copies or substantial portions.

THE FILE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE FILE OR THE USE OR OTHER DEALINGS IN IT.

So a program built with them can have whatever licence its author chooses. The C library linked into it is newlib's, under its own permissive licences (BSD-style).

Code Beside the Examples

- tools/sdk/examples/nuked, Nuked-OPL3, is under the GNU LGPL version 2.1 or later, so a program built with it (oplbench.app and oplplay.app) is under the LGPL as a whole.
- emu8950, used by oplplay, is under the MIT licence.

The Programs Included with the Firmware

- **doom.app** is built from doomgeneric, which is Doom's source code under the GNU GPL version 2. Its source is doomgeneric at the commit named in apps/doom/make.py, with the files of apps/doom.
- **Freedoom** (freedoom1.wad), the game data played by doom.app, is copyright the Freedoom project's contributors, under a BSD 3-Clause licence.
- **scummvm.app** is built from frank-quest, a port of ScummVM, under the GNU GPL version 2 or later. Its source is frank-quest at the commit named in apps/scummvm/make.py, with the files of apps/scummvm.
- **micropython.app** is built from MicroPython (MIT licence) with ulab (MIT) and usqlite with SQLite (public domain), and the Pico Computer 3's modules.

In the Firmware

The firmware's FM synthesiser, which the opl_ and music_ services use, is ymfm, copyright 2021 Aaron Giles, under a BSD 3-Clause licence. Its MIDI instrument bank comes from Freedoom, under a BSD 3-Clause licence. Their notices are in the firmware's release notes.